

StudyStream Solana Mobile Development Manual

For StudyStream
Version 1.0

Based on the current StudyStream architecture and PeerChain protocol documentation.

1. Purpose of This Manual

This guide exists to help junior developers take the existing StudyStream web application and extend it into a Solana mobile-compatible decentralized application (dApp).

The goal is NOT to rebuild the system from scratch.

The goal is to:

- reuse the existing architecture
- preserve the Solana backend
- adapt the frontend for mobile devices
- integrate Solana Mobile Stack properly
- maintain blockchain trust logic across platforms

This manual assumes:

- the web app already exists
- Solana smart contracts/programs already exist or are being developed
- the frontend uses React/Next.js
- the mobile version will target Android first

Reason:

Solana Mobile Stack currently has strongest Android support.

2. What StudyStream Actually Is

Before writing mobile code, understand the product.

StudyStream is NOT:

- just a tutoring app
- just a chat system
- just a blockchain wallet

It is:

a Proof of Learning protocol where educational activity becomes verifiable economic value.

Core loop:

Learning → Proof → Reputation → Funding → Continued Learning

The blockchain layer is the source of truth.

That means:

the mobile app must NEVER bypass blockchain validation logic.

3. High-Level Mobile Architecture

The mobile app should NOT contain all business logic.

The architecture should remain layered.

Recommended structure:

Frontend Layer (React Native)



Mobile Service Layer



Solana RPC / API Layer



Solana Programs (Rust)



On-chain Reputation + Funding Logic

Frontend responsibilities:

- UI
- wallet connection
- session interaction
- displaying reputation

- funding request forms
- voice interactions

Backend responsibilities:

- transaction validation
- reputation calculations
- proof recording
- funding approvals

Never calculate trust-critical values only on-device.

4. Recommended Mobile Stack

For StudyStream mobile:

Layer	Recommended Tech
Mobile Frontend	React Native
Solana Integration	Solana Mobile Stack
Wallet Connectivity	Mobile Wallet Adapter (MWA)
Blockchain SDK	@solana/web3.js
State Management	Zustand or Redux
Backend APIs	Node.js / Supabase
Voice AI	ElevenLabs
Authentication	Wallet-based auth
Storage	AsyncStorage / SecureStore

Why React Native?

Because the existing web frontend is React-based. Reusing mental models speeds development.

5. Solana Mobile Stack (SMS)

The Solana Mobile Stack is the official mobile framework for Solana mobile dApps.

Key components:

Component	Purpose
Mobile Wallet Adapter (MWA)	Connects app to wallet
Seed Vault	Secure private key storage
Solana dApp Store	Distribution
Solana Pay	Payments
Mobile SDKs	Blockchain interaction

Most important rule:

DO NOT build your own wallet.

Use MWA.

Why?

Security.

6. Understanding Mobile Wallet Adapter (MWA)

MWA is the bridge between your mobile app and crypto wallets.

The app requests:

- wallet connection
- transaction signing
- message signing

The wallet handles:

- private keys
- approvals
- secure signing

Flow:

StudyStream App



MWA Request



Wallet App Opens



User Approves



Signed Transaction Returns

Supported wallets include:

- Phantom
- Solflare
- Seed Vault Wallet

7. Seed Vault Explained

Seed Vault is Solana Mobile's secure hardware custody layer.

Important:

Junior devs often misunderstand this.

Seed Vault is NOT for normal dApps.

It is mainly for wallet apps.

StudyStream should use:

- Mobile Wallet Adapter
- existing wallets

NOT direct Seed Vault integration.

Exception:

If later building a native StudyStream wallet.

8. Mobile Project Structure

Recommended folder structure:

```
src/
├── components/
├── screens/
├── navigation/
├── services/
├── |
├── |   ├── solana/
├── |   ├── api/
├── |   └── auth/
├── hooks/
├── store/
├── utils/
├── types/
└── assets/
```

Keep blockchain logic isolated.

Never scatter transaction code everywhere.

9. Setting Up the Mobile Project

Install React Native:

```
npx react-native init StudyStreamMobile
```

Install Solana dependencies:

```
npm install @solana/web3.js
```

```
npm install @solana-mobile/mobile-wallet-adapter-protocol
```

Reference scaffold tutorial:

[Solana React Native First App Tutorial](#)

10. Wallet Connection Flow

Core wallet flow:

1. User taps Connect Wallet
2. MWA opens wallet app
3. User approves connection
4. Public key returned
5. App stores session state

Example architecture:

```
connectWallet()  
  -> authorize()  
  -> getPublicKey()  
  -> saveSession()
```

Never store private keys locally.

Never ask users for seed phrases.

11. Authentication Strategy

Recommended authentication:

Wallet Address = Identity
Signed Message = Authentication

Flow:

- backend sends nonce
- user signs nonce
- backend verifies signature
- JWT/session issued

This prevents fake accounts.

12. Mobile UX Rules

Junior developers usually fail here.

Web UI != mobile UI.

Bad mobile apps:

- overcrowded screens
- tiny text
- desktop layouts squeezed into phones

Rules:

- one primary action per screen
- large tap areas
- bottom navigation
- progressive disclosure
- avoid clutter

StudyStream mobile should feel:

- fast
 - lightweight
 - focused
-

13. Features to Build First

DO NOT build everything immediately.

Build in this order:

Phase 1:

- wallet connection
- profile screen
- reputation display
- session list

Phase 2:

- create session
- join session
- blockchain recording

Phase 3:

- funding requests
- funding approvals
- notifications

Phase 4:

- voice AI
 - AI summaries
 - passive learning tools
-

14. Solana Transaction Design

Every blockchain interaction should be intentional.

Good transactions:

- small
- cheap
- atomic
- verifiable

StudyStream transaction examples:

- record session
- update reputation
- mint achievement NFT
- submit funding request

Avoid:

- unnecessary writes
 - bloated account structures
 - excessive compute usage
-

15. Reputation System Architecture

Reputation should NOT be fully client-side.

Correct flow:

User completes session



Session proof submitted



Backend validates



Solana program updates reputation



Frontend refreshes state

Never trust frontend reputation calculations.

16. Handling Offline Mode

Mobile users lose connectivity constantly.

Support:

- queued actions
- cached session data
- retry transactions
- optimistic UI carefully

Never fake blockchain success.

Always confirm transaction finality.

17. AI Integration

StudyStream already includes AI tutoring and voice systems.

Mobile use cases:

- narrated technical summaries
- voice explanations
- AI funding pitch assistance
- conversational tutoring

Architecture:

Mobile App



AI API Gateway



LLM / ElevenLabs

Do not expose API keys inside the app.

18. Security Rules

Mandatory rules:

NEVER:

- store private keys
- hardcode secrets
- expose RPC admin keys
- trust client reputation values
- expose funding logic client-side

ALWAYS:

- validate server-side
 - verify signatures
 - sanitize input
 - use environment variables
 - separate devnet/mainnet configs
-

19. Android First Strategy

Current Solana Mobile support is strongest on Android.

Recommended approach:

Web App



React Native Android

↓
Optimize architecture
↓
Later evaluate iOS support

Do not try:

- Android
 - iOS
 - desktop
 - PWA
- all simultaneously early on.

That kills small teams.

20. Recommended Development Workflow

Branch strategy:

main
develop
feature/mobile-wallet
feature/reputation-ui
feature/funding-flow

Workflow:

1. frontend branch
2. backend branch
3. QA testing
4. merge review
5. staging deploy

Phoenix should test:

- wallet failures
- rejected signatures
- network disconnects
- invalid transactions
- UI desyncs

As specified in project documentation.

21. Testing Strategy

You must test:

- Android emulator
- physical Android device
- slow internet
- wallet rejection
- transaction timeout
- RPC failures

Use:

- mock wallets
- Solana devnet
- fake transaction simulators

Solana provides mock MWA wallets for development/testing.

22. Common Junior Developer Mistakes

Avoid these:

1. Overengineering immediately
 2. Writing blockchain logic in UI components
 3. Hardcoding wallet addresses
 4. Not handling failed transactions
 5. Ignoring mobile UX
 6. Trusting frontend values
 7. Making giant Solana transactions
 8. Using mainnet too early
 9. Building features before architecture
 10. Mixing business logic with presentation logic
-

23. MVP Mobile Scope

A working mobile MVP should allow:

Connect Wallet



View Profile



Join Session



Record Participation



Update Reputation



Request Funding

If this loop works reliably:
the mobile version succeeds.

Everything else is enhancement.

24. Future Mobile Expansion

Later possibilities:

- biometric authentication
 - mobile push notifications
 - QR session joining
 - Solana Pay support
 - native Seed Vault integration
 - offline-first sync engine
 - AI voice classrooms
 - mobile NFT certificates
-

25. Final Engineering Principle

The mobile app is NOT the product.

The protocol is the product.

Mobile is simply another interface into the StudyStream trust system.

Protect:

- data integrity
- verification logic
- reputation accuracy
- transaction security

If the blockchain trust layer breaks:
the entire economic model collapses.

That is the core responsibility of the engineering team.

Primary References

- [Solana Mobile Docs](#)
- [Mobile Wallet Adapter Docs](#)
- [Seed Vault Docs](#)
- [React Native Solana Tutorial](#)
- StudyStream Protocol Documents
- PeerChain Whitepaper